Prolog

The cut operator

The cut operator

- The cut operator introduces a way to control how the resolution tree is explored
  - It prunes some branches in the tree
  - It eliminates useless tests
  - It cuts a potentially infinite tree
  - It makes a Prolog program more efficient
- Useful when some of the clauses are exclusive
  - i.e. if one clause is true, then the other is necessarily false
  - So no need to explore the next branch when the previous is true
- The symbol ! is used
- The operator elimates some of the unexplored goals of the tree

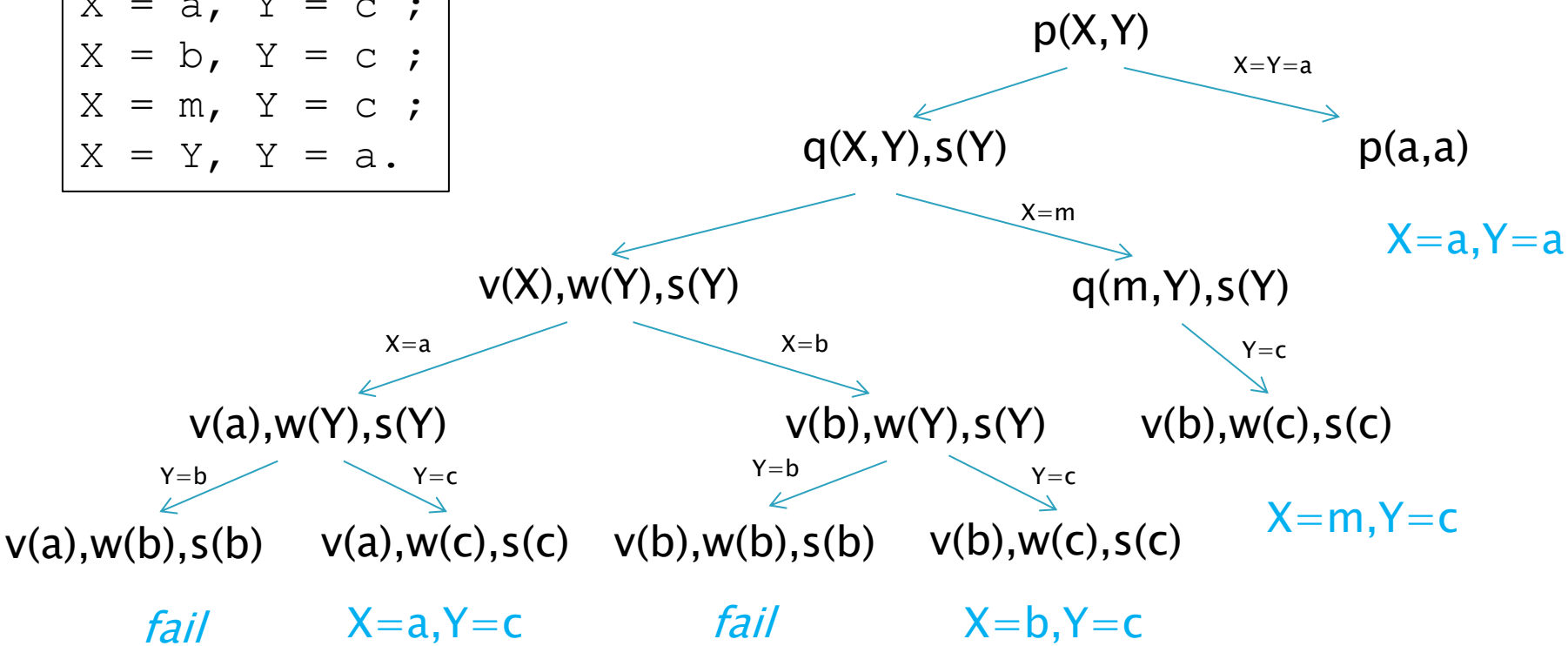
Example 1(no cut)

```
p(X,Y):-q(X,Y),s(Y).
p(a,a).
q(X,Y):-v(X),w(Y).
q(m,Y).
v(a).
v(b).
w(b).
w(c).
s(c).
```

```
?- p(X,Y).
```

Search tree

```
?- p(X,Y).
X = a, Y = c ;
X = b, Y = c ;
X = m, Y = c ;
X = Y, Y = a.
```



The cut operator

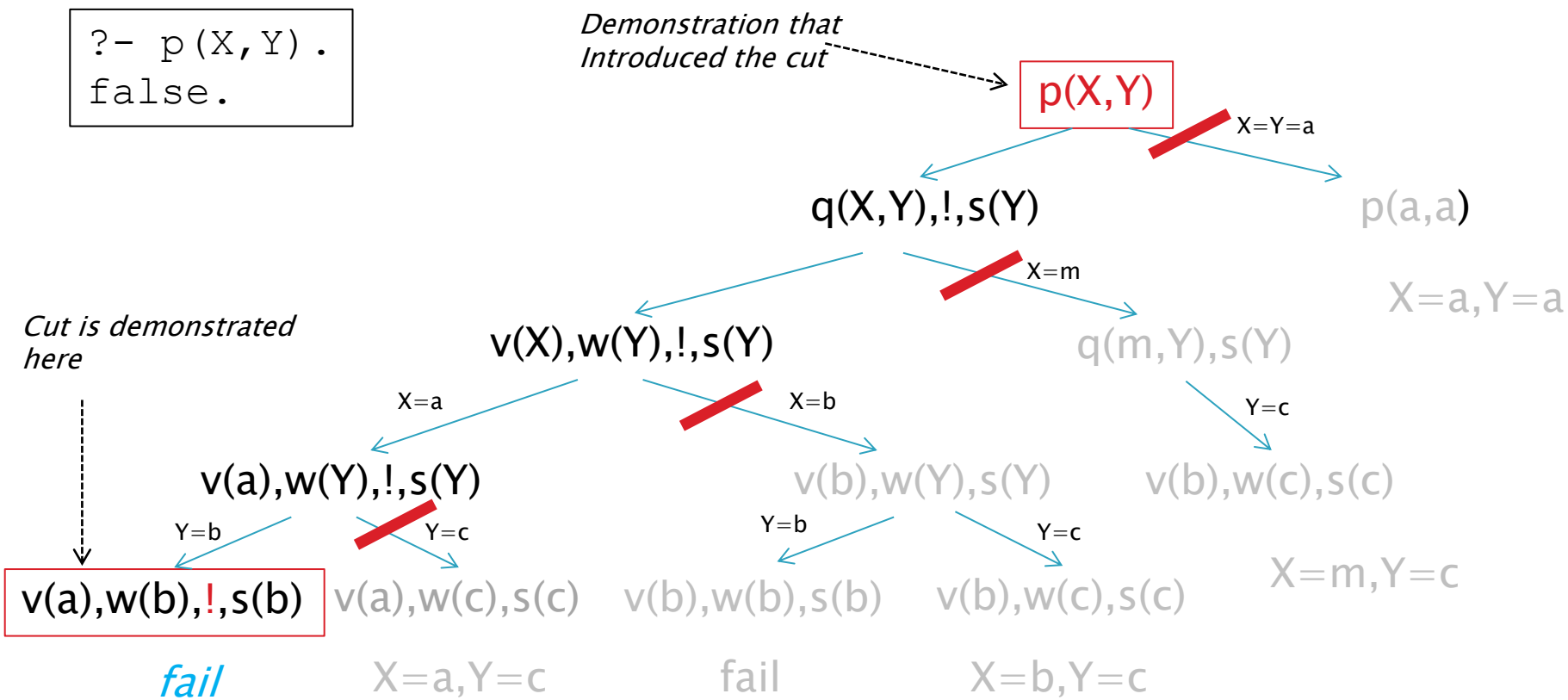
- A cut demonstration is always true
- A cut becomes active when it is demonstrated
- All the demonstrated goals in a clause before the cut is demontrated are frozen
- When backtracking, the exploration resumes at the parent clause
- A cut eliminates the non-explored branches under a node
- The unexplored branches before the cut is introduced and after its demonstration remains available

Example 2

```
p(X,Y):-q(X,Y),!,s(Y).
p(a,a).
q(X,Y):-v(X),w(Y).
q(m,Y).
v(a).
v(b).
w(b).
w(c).
s(c).
```

Search tree

```
?- p(X,Y).
false.
```



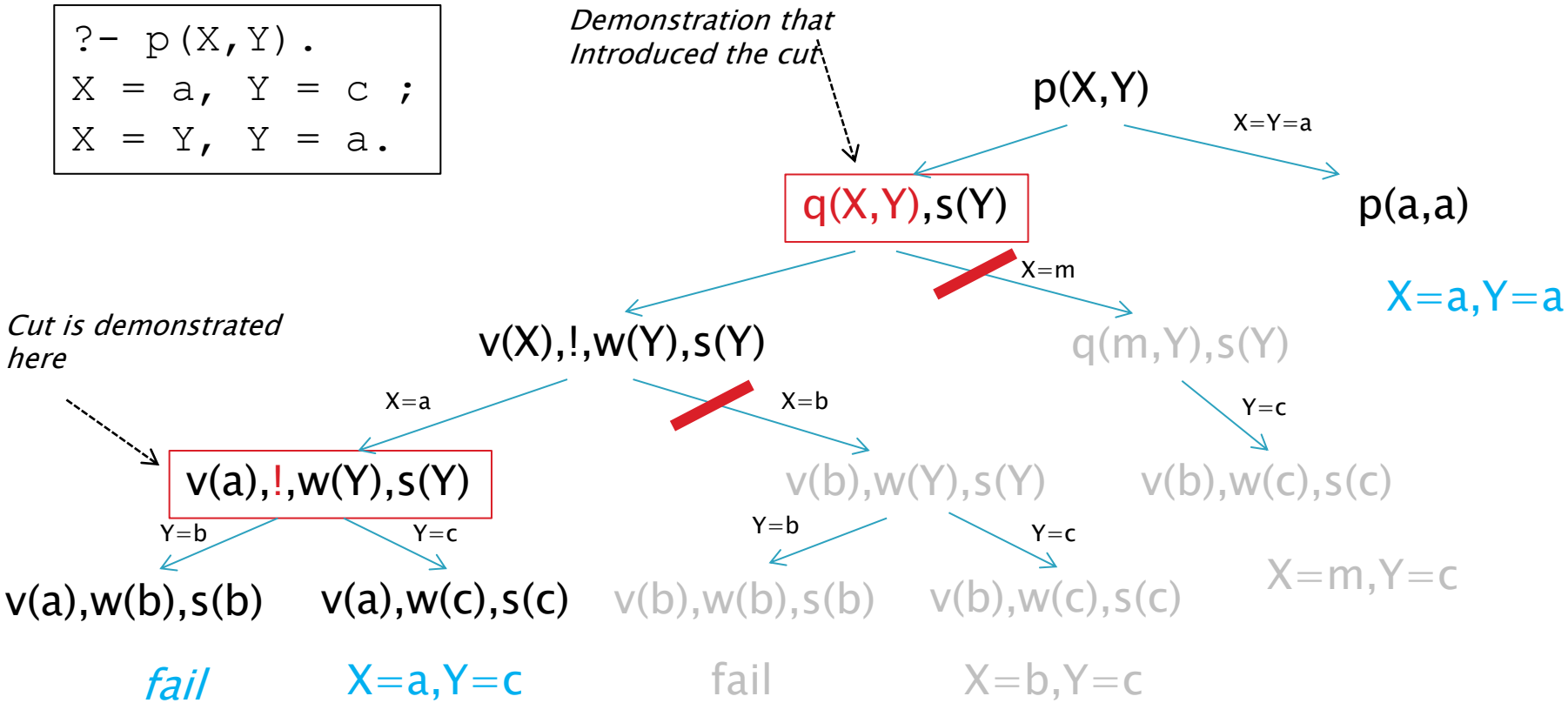
Example 3

```
p(X,Y):-q(X,Y),s(Y).
p(a,a).
q(X,Y):-v(X),!,w(Y).
q(m,Y).
v(a).
v(b).
w(b).
w(c).
s(c).
```

```
?- p(X,Y).
X = a, Y = c ;
X = b, Y = c ;
X = m, Y = c ;
X = Y, Y = a.
```

Search tree

```
?- p(X,Y).
X = a, Y = c ;
X = Y, Y = a.
```



Cut to stop an infinite tree

- Find the integer root of a number
  - It is the largest integer for which its square is still inferior to the number

```
int_root(N, R) :- integer(K), K*K >N, ! , R is K-1.
```

- The generator never stop
  - The cut is introduced to stop the generator
  - This is a *red* cut

Red cut – Green cut

- A red cut modifies the solution space
  - For example by stopping an infinite search
- A green cut avoid the demonstration of clause choice that would have failed anyway
  - Makes the program more efficient
  - Often used in if-then-else patterns

```
d :- a,!,b.
d :- c.
```

  - If `a` then `b` else `c`

If-then-else and green cuts

```
max(X,Y,Y) :- X <= Y.
max(X,Y,X) :- X > Y.
```

```
max(X,Y,Y) :- X <= Y, !.
max(X,Y,X) :- X > Y.
```

```
max(X,Y,Y) :- X <= Y, !.
max(X,Y,X).
```

```
max(X,Y,Z) :- X <= Y, !, Y=Z.
max(X,Y,X).
```

Another example with red cuts

```
p(X,Y) :- q(X), r(X,Y).
p(c,c1).

q(a).
q(b).
r(a,a1).
r(a,a2).
r(b,b1).
r(b,b2).
r(b,b3).
```

Another example with red cuts

```
p(X,Y) :- q(X), r(X,Y), !.
p(c,c1).
```

1 solution

```
p(X,Y) :- q(X), !, r(X,Y).
p(c,c1).
```

2 solutions

```
p(X,Y) :- !, q(X), r(X,Y).
p(c,c1).
```

5 solutions

The fail predicate

- `fail` is a special predicate that is never demonstrable
- Demonstrating a clause that contains a fail always never succeed

The cut-fail

```
admitted(Candidate) :-
    record(Candidate, criminal), !, fail.

admitted(Candidate) :-
    age(Candidate, Age),
    Age > 18, other-condition(Candidate).
```

Used when a demonstration invalidates the other choices

Recap: Why cuts?

- To eliminate clause choices that would certainly fail
- To eliminate useless testing of mutually exclusive clauses
- To obtain only the first demonstrable solution
- To control the demonstration process

Some cut variants

Facts:

holiday(friday).
weather(friday,sunny).
weather(saturday,sunny).
weather(sunday,sunny).
weekend(saturday).
weekend(sunday).

Rules:

picnic(J) :- weather(J,sunny), weekend(J).
picnic(J) :- holiday(J).

Query:

?- picnic(J).

Result:

J=saturday;
J=sunday;
J=friday.

Some cut variants

Facts:

holiday(friday).
weather(friday,sunny).
weather(saturday,sunny).
weather(sunday,sunny).
weekend(saturday).
weekend(sunday).

Rules:

picnic(J) :- weather(J,sunny), !, weekend(J).
picnic(J) :- holiday(J).

Query:

?- picnic(J).

Result:

No.

Some cut variants

Facts:

holiday(friday).
weather(friday,sunny).
weather(saturday,sunny).
weather(sunday,sunny).
weekend(saturday).
weekend(sunday).

Rules:

picnic(J) :- weather(J,sunny), weekend(J), !.
picnic(J) :- holiday(J).

Query:

?- picnic(J).

Result:

J=saturday.

Some cut variants

Facts:

```
holiday(friday).
weather(friday,sunny).
weather(saturday,sunny).
weather(sunday,sunny).
weekend(saturday).
weekend(sunday).
```

Rules:

```
picnic(J) :- !, weather(J,sunny), weekend(J).
picnic(J) :- holiday(J).
```

Query:

```
?- picnic(J).
```

Result:

```
J=saturday;
J=sunday.
```